

CAMPUS ACADEMIC MANAGEMENT SYSTEM

Java-Based Academic Management Project

Project Report

1. Introduction

The Campus Academic Management System is a console-based Java application designed to organize student and teacher academic information. It manages student details, subjects, marks, attendance and performance reports.

2. Problem Statement

Manual management of academic records can be time-consuming and can make it difficult to calculate performance and attendance accurately. The proposed system provides a simple centralized academic workflow.

3. Objectives

Organize academic records; reduce manual calculations; calculate marks percentages and grades; calculate attendance eligibility; generate student performance reports; demonstrate Java Object-Oriented Programming.

4. Functional Requirements

Teacher portal; Student portal; add students; view students and teachers; store subjects and marks; calculate percentages; calculate attendance; calculate grades and pass/fail status; generate reports.

5. Non-Functional Requirements

Usability, reliability, maintainability, performance, validation and scalability. The application uses clear menus, separate classes, input validation and in-memory collections.

6. System Architecture

Main.java controls user interaction and menu flow. AcademicSystem manages collections of students, teachers, subjects, marks and attendance. Domain classes store and process academic information. ReportGenerator and Result provide reporting and result processing.

7. Workflow

Start → Main Menu → Teacher Portal / Student Portal → Select operation → Validate input → Process academic data → Display result/report → Return to menu → Exit.

8. Design / OOP

User is the base class. Student and Teacher inherit from User. Private fields demonstrate encapsulation. displayDetails() is overridden in Student and Teacher. Marks and Attendance validate data using exceptions. ArrayList is used for in-memory storage.

9. Implementation

The project contains 10 Java classes: Main, AcademicSystem, User, Student, Teacher, Subject, Marks, Attendance, Result and ReportGenerator. This modular structure separates responsibilities.

10. Testing

The source files were successfully compiled with `javac *.java` and the application was successfully executed with `java Main`. Important test areas include menu navigation, student details, marks percentage, attendance, result generation and invalid numeric input.

11. Challenges

Handling Java compilation from PowerShell, organizing multiple classes, validating academic values, and maintaining a clear menu-driven workflow were key implementation challenges.

12. Learnings

The project provided practical experience with classes and objects, inheritance, encapsulation, method overriding, ArrayList, exception handling, constructors, validation and modular program design.

13. Future Enhancements

Database connectivity, secure login, update/delete CRUD operations, persistent storage, GUI/web interface, advanced analytics and role-based access can be added in future versions.

14. References

Java documentation and course learning materials.